

TRABAJO EXTRACLASE

INGENIERÍA DE SOFTWARE II

Conectando la IA con el Mundo Real a través de MCP

1. Contexto Académico y Propósito

El **Model Context Protocol (MCP)**, co-desarrollado por Anthropic, surge como un estándar abierto para resolver este aislamiento. MCP define un protocolo de comunicación bidireccional (basado en JSON-RPC) que permite a un **Client** (como Claude Desktop o un IDE) interactuar de forma segura con un **Server** (que expone datos y herramientas específicas).

El **propósito de esta actividad** es que los estudiantes, comprendan de forma práctica la arquitectura de MCP y experimenten cómo este estándar unifica la forma en que los modelos de lenguaje acceden a recursos, ejecutan herramientas (*tools*) y gestionan plantillas de prompts (*prompts*). No buscamos una arquitectura hipercompleja, sino un entendimiento claro de su valor de negocio, viabilidad técnica y versatilidad.

2. Objetivos de Aprendizaje

Al finalizar este trabajo extraclase, el estudiante será capaz de:

1. **Explicar** la arquitectura del Model Context Protocol (Host, Cliente, Servidor) y la diferencia entre *Resources*, *Tools* y *Prompts*.
2. **Evaluar** cómo MCP simplifica los problemas tradicionales de integración de software y desarrollo de agentes de IA en comparación con las integraciones API personalizadas (ad-hoc).
3. **Demostrar o implementar** una integración básica de un LLM con fuentes de datos externas o herramientas operativas mediante MCP.
4. **Comunicar** con rigor técnico y claridad ejecutiva la arquitectura, beneficios y limitaciones de la solución analizada ante un público profesional.

3. Descripción de la Tarea

Cada grupo seleccionará **una de las tres opciones de trabajo** detalladas a continuación. Esta flexibilidad les permitirá enfocar el esfuerzo de acuerdo con sus habilidades de desarrollo actuales, asegurando un aprendizaje significativo en poco tiempo.

Opciones de Trabajo (Seleccionar una):

Opción A: Implementación Práctica de un Servidor MCP Básico

Consiste en programar un servidor MCP simple utilizando uno de los SDKs oficiales (Python o TypeScript/JavaScript). El servidor debe exponer al menos un recurso (*Resource*) o una herramienta (*Tool*) personalizada.

- *Ejemplo:* Un servidor MCP en Python que lea archivos .log locales, calcule estadísticas simples de texto, o consulte una API pública del clima, exponiendo esta funcionalidad como una *Tool* ejecutable por un LLM.

Opción B: Orquestación y Demostración de Herramientas Existentes

Consiste en configurar y desplegar un entorno utilizando servidores MCP existentes en la comunidad (disponibles en repositorios oficiales o en *Smithery.ai*). Los estudiantes deberán conectar un cliente compatible (por ejemplo, Claude Desktop, Cursor, VS Code) con al menos dos servidores MCP funcionales y demostrar la resolución de un problema práctico.

- *Ejemplo:* Conectar Claude Desktop al servidor MCP de *SQLite* para consultar una base de datos local de clientes y, simultáneamente, al servidor MCP de *GitHub* para documentar los hallazgos en un repositorio, todo mediante lenguaje natural.

Opción C: Análisis de Caso de Uso y Arquitectura de Negocio

Orientado a un enfoque de arquitectura y consultoría de software. El grupo deberá seleccionar un problema complejo de la industria (ej. Salud, Finanzas, Logística) y diseñar conceptualmente cómo MCP puede estructurar la solución. Deben detallar el flujo de datos, la definición de los esquemas de las *Tools* y *Resources* requeridos, y las ventajas de seguridad y estandarización que ofrece MCP frente a arquitecturas tradicionales.

4. Requisitos Mínimos

Dependiendo de la opción elegida, el grupo deberá cumplir con los siguientes mínimos para aprobar la actividad:

Requisito	Opción A (Implementación)	Opción B (Demostración)	Opción C (Caso de Uso/Arquitectura)
Componentes MCP	Definir claramente la interfaz JSON-RPC	Identificar con precisión los servidores, clientes y hosts utilizados.	Definir formalmente los esquemas JSON de las <i>Tools</i> y <i>Resources</i> propuestos.

Material de Soporte	Código fuente estructurado y documentado (GitHub).	Archivo de configuración del cliente MCP (ej. <code>claude_desktop_config.json</code>).	Diagrama de secuencia o arquitectura que ilustre el flujo de mensajes MCP.
Demostración	Demo en vivo o video corto (máx. 2 min) del modelo consumiendo el servidor.	Demo en vivo usando un cliente compatible interactuando con los dos servidores.	Simulación paso a paso del flujo de prompts, contextos y respuestas del sistema.

5. Ejemplos de Áreas y Servidores de Inspiración

Como guía y a manera de ejemplo, pueden explorar los siguientes ecosistemas de integración compatibles con MCP:

- **Sistemas de Archivos:** Lectura, escritura e inspección segura de directorios locales.
- **Bases de Datos:** Servidores MCP para SQLite, PostgreSQL, MySQL o Redis (permiten al modelo consultar esquemas y ejecutar queries).
- **Herramientas de Desarrollo:** Integración con GitHub/GitLab (creación de issues, pull requests, revisión de código).
- **Productividad y Comunicación:** Conexión con Slack, Google Drive, Notion o Jira.
- **APIs Web & Búsqueda:** Servidores para buscar en Google, consultar Wikipedia, obtener datos meteorológicos o interactuar con APIs de mapas.

6. Entregables Esperados

Antes de iniciar la siguiente sesión de clase, cada grupo deberá entregar al correo institucional un enlace a un repositorio de GitHub (para Opción A y B) o un documento técnico breve (para Opción C). Los entregables obligatorios son:

1. **Código / Configuración / Reporte:**
 - *Opción A:* Código del servidor e instrucciones de instalación (README.md).
 - *Opción B:* Archivos de configuración aplicados y capturas de pantalla de la interacción.
 - *Opción C:* Documento en PDF con el análisis arquitectónico, diagramas y especificación de servicios.
2. **Presentación:** Diapositivas preparadas para la exposición en clase.

7. Estructura Sugerida para la Exposición en Clase

Cada grupo dispondrá de un **máximo de 8 minutos** para exponer su trabajo, seguidos de 2

minutos de preguntas. Les sugiero la siguiente estructura para optimizar el tiempo:

- **Contextualización del Problema.** ¿Qué problema o necesidad de integración se identificó? ¿Por qué los métodos tradicionales de API o LLMs estándar tienen dificultades para resolverlo de forma sencilla?
- **Arquitectura de la Solución.** Explicar el flujo del sistema: ¿Quién es el Host, el Cliente y el Servidor MCP? ¿Qué recursos, herramientas o esquemas se definieron?
- **Demostración Práctica.** Demo en vivo o video pregrabado donde se vea la interacción en tiempo real del modelo de IA interactuando con la herramienta o servicio externo mediante MCP.
- **Lecciones Aprendidas y Conclusiones.** ¿Qué ventajas identificaron en MCP (ej. tipado estricto, estandarización)? ¿Qué limitaciones o retos técnicos experimentaron?

8. Criterios de Evaluación

La calificación final de la actividad se calculará mediante la siguiente ponderación de criterios:

- **Rigor Técnico y Comprensión del Protocolo (40%):**
 - Demuestra un entendimiento claro de cómo fluyen los datos bajo la especificación MCP.
 - Define o utiliza correctamente conceptos clave: Clients, Servers, JSON-RPC, Tools, Resources, Prompts.
- **Calidad de la Demostración / Solución Propuesta (30%):**
 - La demo es funcional, estable y resuelve de forma inteligente el problema planteado, o bien el diseño arquitectónico propuesto en el Track C está detallado de forma impecable y viable.
- **Estructura y Claridad de la Exposición (20%):**
 - Presentación ágil, profesional, que se ajusta estrictamente al límite de tiempo (8 min).
 - Uso adecuado del lenguaje técnico de ingeniería de software e inteligencia artificial.
- **Documentación y Entregables (10%):**
 - El repositorio o reporte técnico es autoexplicativo, limpio y ordenado, permitiendo replicar el ejercicio de manera ágil.

9. Recomendaciones

Algunas recomendaciones clave para maximizar su eficiencia:

1. **No reinventen la rueda:** Si seleccionan la Opción A, utilicen los SDKs oficiales de MCP en Python o TypeScript. Estos ya resuelven todo el manejo del protocolo subyacente por debajo, permitiéndoles concentrarse en programar únicamente la lógica de sus *Tools* y *Resources*.
2. **Aprovechen la documentación oficial:** Visiten modelcontextprotocol.io para comprender rápidamente la especificación mediante esquemas ilustrados y consultar guías rápidas de inicio (*Quickstarts*).

3. **Para la Opción B:** Herramientas como Claude Desktop o Cursor facilitan enormemente el testeado local. Pueden usar proyectos listos de la comunidad de GitHub para arrancar en minutos.
4. **Enfoque en el MVP:** Diseñen un Producto Mínimo Viable extremadamente simple. Es preferible un servidor MCP con una única herramienta funcional que haga algo básico de manera robusta, que intentar construir una app compleja que falle en vivo durante la presentación.